

Course Name: Principles of Programming Languages

Class: SE Computer Engineering

Course In-charge: Payal Rajput

Course Outcome

On completion of the course, learners should be able to

CO1: Make use of basic principles of programming languages.

CO2: Develop a program with Data representation and Computations.

CO3: Develop programs using Object Oriented Programming language : Java.

CO4: Develop application using inheritance, encapsulation, and polymorphism.

CO5: Demonstrate Multithreading for robust application development.

CO6: Develop a simple program using basic concepts of Functional and Logical programming
paradigm.

Progressive Education Society's
Modern College of Engineering, Pune-05.
DEPARTMENT OF COMPUTER ENGINEERING

Unit-II
Structuring the Data, Computations and Program

On completion of the unit, learners should be able to

CO2: Develop a program with Data representation and Computations..

Elementary Data Types

- Elementary data types, also known as primitive data types, are the basic types of data that serve as the building blocks for more complex data structures in programming.
- Data Types specify the different sizes and values that can be stored in the variable.
- There are 2 types of data types:

1. Primitive
2. Non-Primitive

Primitive data types

- **Primitive data types** are the most basic types of data that are directly supported by a programming language.
- There are mainly 8 primitive data types:

Integer	Character
Float	double
Boolean	Byte
Short	Long

Character String Types

- A character string is a collection of characters that are often specified by enclosing the characters in a single(‘’) or double quotes(“”).
- Example: char[] ch={'j','a','v','a','t','p','o','i','n','t'};

  String s=**new** String(ch);

  is same as:

  String s="javatpoint";

User Defined Ordinal Types

- **User-defined ordinal types** are types created by the programmer that represent a set of ordered values.
- Ordinal types are typically used to represent a sequence of values where the order matters.

Why Use User-Defined Ordinal Types?

1. Improved Code Readability
2. Type Safety
3. Better Maintenance

User Defined Ordinal Types

❖ **Example: In C:**

```
enum Day { Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday };
```

- By default, the first value (Sunday) gets the value 0, the next (Monday) gets 1, and so on. You can explicitly assign values if needed

```
enum Day { Sunday = 1, Monday = 2, Tuesday = 3, Wednesday = 4, Thursday = 5, Friday = 6, Saturday = 7 };
```

However, they still represent a fixed set of ordered constants.

User Defined Ordinal Types

In Java:

```
public enum Day {
   SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}
```

- In Java, enums are more sophisticated and can have methods, fields, and constructors.

Progressive Education Society's
Modern College of Engineering, Pune-05.
DEPARTMENT OF COMPUTER ENGINEERING

Types of Arrays

- An **Array** is a data structure that stores a fixed-size sequence of elements of the same data type.
- Arrays can be categorized in various ways based on their dimensionality and how they are used. Here are the most common types:

1. One-dimensional
2. Two-dimensional
3. Multi-dimensional
4. Dynamic Array
5. Associative Array
6. Array of Array

One Dimensional:

```
int arr[5] = {1, 2, 3, 4, 5};
```

Multi-Dimensional:

```
int arr[2][2][2] = {{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}};        // 3D array
```

Dynamic Array:

A **Dynamic Array** is an array that can grow or shrink in size during runtime.

Two-Dimensional Array:

A **Two-Dimensional Array** is an array of arrays, where each element of the array is itself an array.

Example: int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};

Array of Array:

An **Array of Arrays** (also called a **Jagged Array**) is an array where each element is an array, and the sub-arrays can have different lengths.

Example:
```
int[][] jaggedArray = new int[2][];          // Jagged Array with 2 rows
jaggedArray[0] = new int[] {1, 2, 3};        // 1st row with 3 elements
jaggedArray[1] = new int[] {4, 5};           // 2nd row with 2 elements
Console.WriteLine(jaggedArray[0][1]);        // Access element at row 0, column 1
```

Associative Array:

- An associative array stores the set of elements in the form of (key, value) pairs.This is often referred to as a **map**, **dictionary**, or **hash table** in different programming languages.

Common Use Cases:

- Storing user information (like name, age, etc.) where keys are field names.
- Mapping unique identifiers (like IDs) to objects or values.
- Efficiently counting occurrences of items in a collection like word .

- Example:

```
let associativeArray = { "name": "John", "age": 25 };
console.log(associativeArray["name"]);                    // Output: John
```

Type	Description	Use Cases
Dynamic Array	Resizable array that can grow/shrink during runtime	Lists of varying size, dynamic data structures
Two-Dimensional Array	Array of arrays, typically representing a matrix or grid	Tables, grids, 2D game boards
Associative Array	Key-value pairs for mapping keys to values	Lookup tables, dictionaries, mapping relationships
Multi-Dimensional Array	Arrays with more than two dimensions	3D objects, time series, complex datasets
Array of Arrays	Array where each element is an array (different lengths allowed)	Jagged grids, non-uniform data



Record types

- A record (also known as a struct in languages like C) is a composite data type that groups variables of different types into a single unit.
- Each element within a record is called a **field** or **member**.
- Fields can be of different data types (e.g., integer, float, string).

Record types

- Example:
```
struct Employee {
    int id;
    char name[50];
    float salary;
};
```

Union types

- A union is a data type where all members share the same memory location. At any point, only one member of the union can hold a value.
- More memory efficient compared to records.
- The size of the union is determined by the size of its largest member.

Example:

```
union Data {
        int i;
        float f;
        char c;
};
```



Pointer

- A pointer is a variable that stores the address of another variable.
- Provides direct memory access.
- Allows dynamic memory allocation and efficient data manipulation.
- Example: In C

```
int a = 5;
int *p = &a;
```



Reference

- A reference is an alias for another variable, and unlike pointers, a reference cannot be null or changed to refer to a different variable.
- A reference must be initialized at the time of creation.
- Simplifies pointer manipulation by avoiding dereferencing.
- Example:

```
int x = 10;
int &ref = x;          // ref is a reference to x
```

Feature	Record	Union	Pointer	Reference
Memory Usage	Separate memory for each field	Shares memory for all members	Points to a memory address	Refers directly to data
Access Method	Direct access to fields	Access only one member at a time	Dereferencing pointer	Direct access to the variable
Flexibility	High	Limited to one member at a time	Very flexible	Less flexible (fixed reference)
Use Cases	Structuring related data	Memory-efficient data storage	Dynamic memory allocation	Simplifying data handling

```
struct Data {
        union {
                int i;
                float f;
            };
          char name[50];
       };
Data *ptr;
Data dataInstance;
ptr = &dataInstance;
 ptr->i = 100;
```

Expression & Assignment Statements:

Arithmetic Operators:

- Arithmetic expressions involve operators like +, -, *, /, and % that perform mathematical calculations.
- Operators have a specific order of evaluation (precedence).

Overloaded Operator:

- It is a methodology by which you can redefine the operator behaviour according to your custom need(e.g., classes).
- Operators are redefined to work with objects of a class.

Type Conversions:

- Type conversion refers to changing one data type to another (e.g., from int to float).

Implicit Type Conversion:

- Done automatically by the compiler when no loss of data occurs.
- Example:

```
float x = 5;      // Implicit conversion from int to float
```

Explicit Type Conversion (Casting):

- Manually forcing conversion using cast operators.
- Example:

```
float f = 5.7; int x = (int)f;          // Explicit conversion from float to int
```

Relational & Boolean Expressions:

Used to compare values (e.g., ==, !=, >, <, >=, <=).

Used for logical operations, returning true or false.

Logical Operators: &&, ||, !



Short Circuit Evaluation:

- Short-circuit evaluation refers to the process of evaluating a logical expression from left to right, stopping as soon as the result is determined.

Assignment Statements:

- An assignment statement assigns a value to a variable using the = operator.

Mixed Mode Assignment:

- Mixed mode assignment occurs when operands of different types are involved in an assignment or expression.

- Type conversion happens implicitly/explicitly to ensure compatibility between operand types.

Example:

```
int a = 5;
float b = 2.5;
double result = a + b;  // Implicit conversion from int to float and then to double
```



Statement Level Control Statement:

Selection statement:

- Selection statements allow the program to make decisions based on certain conditions.
- They are used to control the flow of execution depending on whether a condition is **true** or **false**.

if Statement, if-else Statement, if-else if-else Ladder, switch Statement

Nested Selection Statement:

- Selection statements can be nested inside one another to handle more complex conditions.



Iterative Statements:

- Iterative (loop) statements are used to repeat a block of code multiple times as long as a condition is true.
- There are three primary loop types in most programming languages:

  for loop, while loop, do-while loop

Infinite Loops:

- An infinite loop occurs when the loop condition is always true or never properly updated, causing the loop to run indefinitely.



Unconditional Branching :

- Unconditional branching allows the program to jump to a different part of the code without any conditions.
- It is typically used for **breaking** loops or **jumping** to specific locations in the code.

`break` **Statement,** `continue` **Statement,** `goto` **Statement**



Sub Program

- A subprogram is a block of code designed to perform a specific task.
- It can be invoked by the main program or other subprograms.

Types of Subprograms**:**

- **Functions**: Return a value and can be used as part of expressions.
- **Procedures**: Do not return values but perform actions.



Fundamentals of Sub Program

Structure: Contains the subprogram name and parameters (if any) & actual instructions to be executed.

Control Flow: A subprogram is invoked (called) from a point in the program & after execution, control returns to the calling code.

Activation Record: A data structure used to manage subprogram calls (stores local variables, parameters, & return addresses).



Design Issues for Sub Program

- Parameter Passing
- Local Vs Global Variables
- Recursion
- Overloading

Local Referencing Environment

- Local variables subprograms can define their own variables, thereby defining local referencing environments.
- A set of variables and data structures that are available to a subprogram during its execution.
- Local variables are created on entry and destroyed on exit.
- They have limited scope and lifetime.



Parameter Passing Method

1. Pass-by-Value
2. Pass-by-Reference
3. Pass-by-Value-Result
4. Pass-by-Name



Abstract Data types and encapsulation Construct

- An **Abstract Data Type (ADT)** is a model for data structures that defines the behavior of data and operations on the data without specifying the implementation details. i.e; Data is hidden (abstraction).

  Example: **List**, **Queue**, **Stack**

- Design Issues for Abstraction:

1. Data Hiding
2. Interface design
3. Efficiency
4. Error Handling



Parameterized Abstract data type

- A **Parameterized Abstract Data Type (PADT)** is a data structure or a set of operations that are defined in a generic manner, where certain characteristics are specified by parameters (e.g., data type).

- PADTs enable the creation of flexible, reusable, and type-safe data structures, enhancing code efficiency and maintainability.



Key features of PADTs:

1. **Parameterization**: The primary feature of a PADT is that it allows parameters to be passed to define the type of data it will work with.
2. **Abstract Interface**: PADTs define a set of operations (such as insert, delete, or search), but the specific implementation details are hidden
3. **Reusability**: The use of parameters allows a single PADT to be adapted for different use cases, rather than needing separate data types for each specific scenario.
4. **Generics**: In programming languages like Java, C++, or Python, parameterized types are often referred to as **generics**.



Naming Encapsulation

- Encapsulation is the OOP concept of hiding an object's internal state and exposing only a controlled interface.
- These names help developers understand how the data inside an object should be interacted with and maintained.

1. Private Members (Attributes): Store internal state, not accessible from outside the class.

Naming Convention:

- **CamelCase** or underscore (_) prefix.
- Example: _size, private int count;

2. Getter Methods: Provide read-only access to private attributes.

Naming Convention:

- **get** prefix for regular attributes.
- **is** or **has** prefix for booleans.
- Example: getSize(), isEmpty(), hasItems()

3. Setter Methods: Provide write access to private attributes with validation.

Naming Convention:

- **set** prefix for attributes.
- Example: setSize(int size), setAge(int age)

4. Constructor Methods: Initialize the object's state.

- **Naming Convention**:
  - Same name as the class, no return type.
  - Example: public Person(String name, int age)

5. Private Accessors (Optional): Internal methods to manipulate or calculate data.

- **Naming Convention**:
  - **Private** and camelCase.
  - Example: private void calculateSize()



6. Public Interface Methods (Business Logic)

- **Purpose**: Expose object behavior to the outside world.
- **Naming Convention**:
  - Action verbs like add, remove, update.
  - Example: addItem(T item), calculateTotal()

THANK YOU !!